FNES = Finite Non Empty Set

Grammar & Languages

In theory of compitation, grammar can be defined as a finite set of formal rules that are generating syntactically correct sentences.

A grammar is defined as a four tuples:

$G = <V, E, P, S>$ where,

V = FNES of variables (non terminals)

$T/\Sigma/E$ = FNES of terminals

P = FNES of production rules

S = Start symbol.

V is denoted by uppercase letters

E is denoted by lowercase letters

Production rules are used for replacing variables with other variables, terminals or a combination.

Q form the tuples for the following grammar

$S \rightarrow aA / bB$; $A \rightarrow a$; $B \rightarrow b$

$V = \{S, A, B\}$

$E = \{a, b\}$

$P = \{S \rightarrow aA, S \rightarrow bB, A \rightarrow a, B \rightarrow b\}$

$S = S$

Q Write the language for the given grammar.

$S \to aS$, $S \to a$

$S \to aS \to a(aS) \to aa(aS) \to \ldots$

$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$

$a \quad a(a) \quad aa(a) \quad aaa(a)$.

$L = \{a, aa, aaa, \ldots\} = a^+$

$S \to aS/\varepsilon \Rightarrow L = a^*$

Q Write a grammar for the given languages

(i) $L = a^*b^*$

$S \to AB$, $A \to aA/\varepsilon$, $B \to bB/\varepsilon$

OR

$S \to aS/Sb/\varepsilon$ $L = \{\varepsilon, a, b, ab, aab, abb, \}$

(ii) $L = (a+b)^*$

$S \to aS/bS/\varepsilon$ $L = \{\varepsilon, a, b, ab, ba, \ldots\}$

(iii) $L = a^n b^n$ for $n \geq 0$

$S \to aSb/\varepsilon$ $L = \{\varepsilon, ab, aabb, aaabbb, \ldots\}$

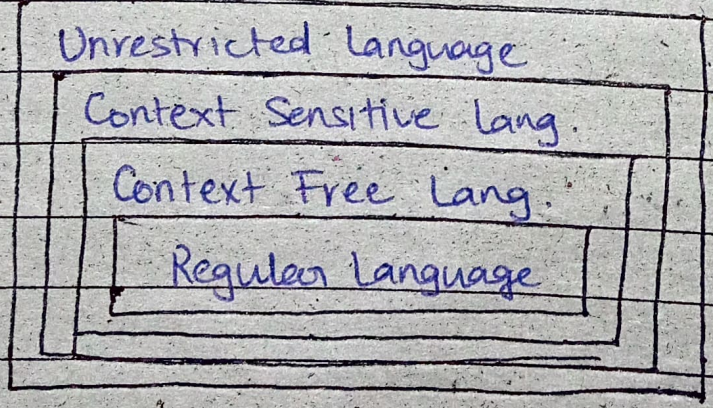
Chomsky Classification of formal languages

- **Unrestricted Language (Type 0)**
  gen. form: $\alpha \longrightarrow \beta$ $\quad \alpha, \beta \in (V+\Sigma)^*$

- **Context Sensitive Lang. (Type 1)**
  gen form: $\alpha A \beta \rightarrow \alpha \lambda \beta$ $\quad \alpha, \lambda, \beta \in (V+\Sigma)^*$
  $A \in V$
  $\alpha \neq \varepsilon \;||\; \beta \neq \varepsilon$

- **Context Free Lang. (Type 2)**
  gen form: $A \rightarrow \lambda$ $\quad \lambda \in (V+\Sigma)^*$
  $A \in V$

- **Regular language (Type 3)**
  gen form: $A \rightarrow aB \mid Ba \mid a \mid \varepsilon$ $\quad a \in \Sigma ; A, B \in V$



By looking at the production rules, we can conclude its grammar type, but not its language type.

eg: $A \rightarrow AB \mid aA \mid \varepsilon$, $B \rightarrow bB \mid \varepsilon$

by looking at the production rule we can conclude that this grammar is of the context free type. But if we look closely, its language is not context free language, but a regular language, infact it is $L = a^*b^*$

Derivation of strings from a Grammar

To derive a string from a grammar we apply some or all of its production rules one after another until we end up with a combination of only terminals (i.e., no variables)

for eg: From the grammar $< S \rightarrow aS \mid Sb \mid \varepsilon >$ we derive the string "aab" in the following ways.

S	OR	S	OR	S
→ aS		→ aS		→ Sb
→ aaS		→ aSb		→ aSb
→ aaSb		→ aaSb		→ aaSb
→ aaεb		→ aaεb		→ aaεb
→ aab		→ aab		→ aab

Left Most Derivation (LMD): In this we substitute the left most variable present at any intermediate step.

Right Most Derivation (RMD): In this we substitute the right most variable present at any intermediate step.

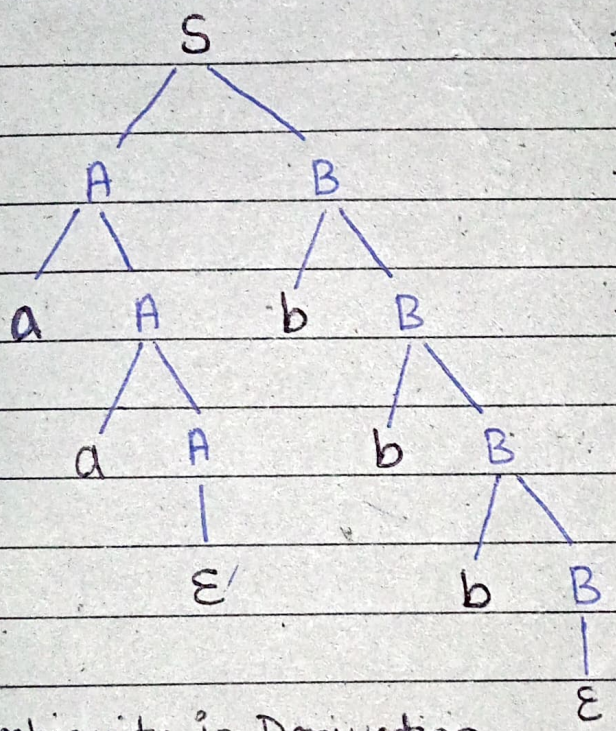
eg: Grammar : $S \rightarrow AB$
$A \rightarrow aA \mid \varepsilon$
$B \rightarrow bB \mid \varepsilon$
String : "aabbb"

LMD	RMD	Random
S	S	S
→ AB	→ AB	→ AB
→ aAB	→ AbB	→ AbB
→ aaAB	→ AbbB	→ aAbB
→ aaεB	→ AbbbB	→ aAbbB
→ aaB	→ Abbbε	→ aaAbbB
→ aabB	→ Abbb	→ aaAbbbB
→ aabbB	→ aAbbb	→ aaεbbbB
→ aabbbB	→ aaAbbb	→ aabbbB
→ aabbbε	→ aaεbbb	→ aabbbε
→ aabbb	→ aabbb	→ aabbb

Derivation Tree = Parse Tree

Derivation Tree : It is a graphical representation of the derivation of a string from a grammar.

for example the derivation tree for the prev. derivation of aabbb



Root node is the start symbol

Intermediate nodes are Variables

Leaf nodes are Terminals

Derivation tree of a string is unique*. whether you do LMD or RMD.

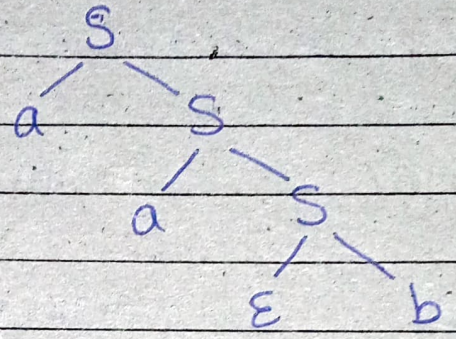
*Ambiguity in Derivation

We just read that the derivation tree of a string is unique for a particular grammar, but sometimes it is not the case. Sometimes a grammar allows multiple D-trees for string to exist. These grammars are known as Ambiguous Grammars.

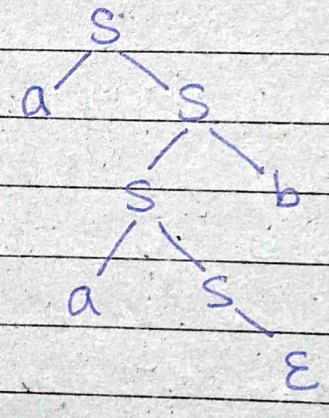
If there exist multiple derivation trees for even a single string, the whole grammar will be labelled as Ambiguous.

eg: Grammar : $S \rightarrow aS / Sb / \varepsilon$
String : "aab"

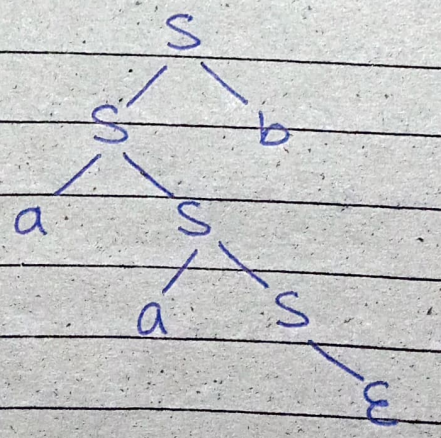
① S
→ aS
→ aSb
→ aaSb
→ aaεb
→ aab



② S
→ aS
→ aSb
→ aaSb
→ aaεb
→ aab



③ S
→ Sb
→ aSb
→ aaSb
→ aaεb
→ aab



hence this grammar is Ambiguous.

Page No.
Date

Simplification of Grammar

A grammar may consists of some extra variables or production rules that unnecessary increase the length of the parse tree. Simplification of grammar means changing the production rules in such a way that reduces the redundant substitutions and hence reduces the length of derivations and D-trees.

Steps of simplification

1. Elimination of variables that do not produce terminals.
2. Elimination of variables that do not appear in derivation.
3. Elimination of chain productions.
4. Elimination of Epsilon productions.

~~Elimination of~~

1) Variables that donot produce terminals

step 1: make $W_1 = \{X \mid X \to \Sigma^*, X \in V\}$

step 2: make $W_{i+1} = \{X \mid X \to (W_i \cup \Sigma)^*, X \in V\} \cup W_i$

step 3: repeat step 2 while $(W_{i+1} \neq W_i)$

new Grammar = $\langle V', \Sigma, P', S \rangle$

where, $V' = W_F$

$P' = P \cap [V' \to (V' \cup \Sigma)^*]$

W_F is set which is obtained at the end of step 3 i.e. the final form of W_i

Page No
Date

eg: $G = S \to AB \mid xAZ \mid Mz$

$A \to aB \mid SA$

$B \to [illegible] \mid SB \mid Xb$

$X \to xY$

$Y \to Zy$

$Z \to z$

$C \to XMz$

$W_1 = \{Z\}$ [$\because Z = z$]

$W_2 = \{Z, Y\}$ [$\because Y = Zy$]

$W_3 = \{Z, Y, X\}$ [$\because X = xY$]

$W_4 = \{Z, Y, X, B\}$ [$\because B = Xb$]

$W_5 = \{Z, Y, X, B, A\}$ [$\because A = aB$]

$W_6 = \{Z, Y, X, B, A, S\}$ [$\because S = AB$]

$W_7 = \{Z, Y, X, B, A, S\}$

since, $W_6 = W_7$

hence, $V' = W_6 = \{$[illegible]$\}$

$G' = < V' = \{S, A, B, X, Y, Z\}$

$\Sigma = \{a, b, x, y, z\}$

$P' = \{ S \to AB \mid xAZ \;,\; X \to xY$

$A \to aB \mid SA \;,\; Y \to Zy$

$B \to SB \mid Xb \;,\; Z \to z \}$

$S = S$

$>$

Page No.
Date

2. Elimination of symbols that do not appear in any derivation

step 1: make $W_0 = \{S_t\}$ i.e. the start symbol

step 2: make $W_{i+1} = \{\lambda \mid W_i \to \alpha \text{ and } \lambda \in \alpha\} \cup W_i$

step 3: repeat step 2 while $(W_{i+1} \neq W_i)$

new Grammar $= \langle V', \Sigma', P', S \rangle$

where, $V' = V \cap W_F$

$\Sigma' = \Sigma \cap W_F$

$P' = P \cap [V' \to (V' \cup \Sigma')^*]$

eg: $G = S \to AB$

$A \to aA \mid a$

$B \to bB \mid b$

$C \to cC \mid c$

$W_0 = \{S\}$ [$\because S = S_t$]

$W_1 = \{S, A, B\}$ [$\because S \to AB$]

$W_2 = \{S, A, B, a, b\}$ [$\because A \to a$ & $B \to b$]

$W_3 = \{S, A, B, a, b\}$

since $W_3 = W_2$

$V' = V \cap W_3 = \{S, A, B\}$ // C is eliminated

$\Sigma' = \Sigma \cap W_3 = \{a, b\}$ // c is eleminated

$P' = \{S \to AB, A \to aA \mid a, B \to bA \mid b\}$

$S_t = S$

// $C \to cC \mid c$ is elimated

Page No
Date

3. Elimination of chain productions.

For all variables V_j in V

step 1 : Identify the chain

1.1 : make $W_1(V_j) = \{V_j\}$

1.2 : make $W_{i+1}(V_j) = W_i(V_j) \cup \{X \mid W_i(V_j) \to X \text{ and } X \in V\}$

1.3 : repeat step 1.2 while $(W_{i+1}(V_j) \neq W_i(V_j))$.

step 2 : Compress the chain

2.1 : if for some $V_k \in W_i(V_j)$ there exists a rule

$V_k \to \alpha \mid |\alpha| > 1$ or $\alpha \in \Sigma^*$

add a new production to G

$V_j \to \alpha$

2.2 : After you end up doing step (2.1) for all V_j remove all the productions of the form

$V_j \to X. \mid X \in V$

eg: $G = \{ S \to A \mid aA$
$A \to B$
$B \to C \mid bB$
$C \to d \}$

Chains found:

$S \to A \to B \to C \to d$
$S \to A \to B \to bB$
$A \to B \to C \to d$
$A \to B \to bB$
$B \to C \to d$

Reduction

$S \to d$
$S \to bB$
$A \to d$
$A \to bB$
$B \to d$

new $G' = \{ S \to aA \mid bB \mid d$
$A \to bB \mid d$
$B \to bB \mid d$
$C \to d \}$

if there exists a production of the form $V_i \to V_j$, there exists a chain.

Page No.
Date

4. Elimination of NULL / ε productions

step1: make $W_1 = \{X \mid X \to \varepsilon \ \& \ X \in V\}$

step2: make $W_{i+1} = \{X \mid X \to W_i^* \ \& \ X \in V\} \cup W_i$

step3: repeat step2 while $W_{i+1} \neq W_i$

step4: the final set W_F so obtained, is the set of all variables that produce ε (directly or indirectly)
now, if $X \in W_F$ has a production of form
$X \to V_1 V_2 V_3 \ldots V_N$; add the new productions of the form: $X \to V_1$, $X = V_2$, $X = V_N$,
$X \to V_1 V_2$, $X \to V_1 V_3$, $X = V_1 V_N$, $X = V_2 V_N$
$X \to V_1 V_2 V_3$, $X \to V_1 V_2 V_4$, ...
i.e. all possible combinations of RHS (with or without each variable)

step 5: Now remove all the productions of the form $V \to \varepsilon$, except for the $S \to \varepsilon$ in case ε belongs to the language of this grammar.

eg: $G = \{ S \to AB$
$A \to a \mid \varepsilon$
$B \to b \mid \varepsilon \}$

$\Rightarrow$

$G' = \{ S \to AB \mid A \mid B \mid \varepsilon$
$A \to a$
$B \to b \}$

$W_F = \{A, B, S\}$

$S \to AB$	$A \to \varepsilon$ ✗
$S \to A$	$B \to \varepsilon$ ✗
$S \to B$	$S \to \varepsilon$ ✓
$S \to \varepsilon$	[$\because \varepsilon \in L$]

Page No.
Date

Chomsky Normal Form (CNF)

A CFG is in CN form iff all productions are of type

$V \rightarrow VV$ or $V \rightarrow \Sigma$

Steps for converting a CFG to its CN form.

step 0: Simplify the grammar

step 1: Replace the combination of variables & terminals (in RHS) with a combination of variables only.

step 2: Reduce the size of the RHS to max 2 variables using staircase approach.

eg: $S \rightarrow aSb \mid aA \mid bB$

$A \rightarrow aA \mid aB$

$B \rightarrow bB \mid b$

(already simplified)

As the grammar is already simplified we will skip step 0.

step 1:

$S \rightarrow XSY \mid XA \mid YB$

$A \rightarrow XA \mid XB$

$B \rightarrow YB \mid b$

$X \rightarrow a$

$Y \rightarrow b.$

now all productions are of type

$V \rightarrow VV$

or, $V \rightarrow \Sigma$

except, $S \rightarrow XSb$

so will redixe its RHS to max two variable using staircase approach

Page No.
Date

Step 2:

$S \rightarrow MY \mid XA \mid YB$

$M \rightarrow XS$

$A \rightarrow XA \mid XB$

$B \rightarrow YB \mid b$

$X \rightarrow a$

$Y \rightarrow a$

now all productions are of type

$V \rightarrow VV$

or $V \rightarrow \Sigma$

hence this grammar is in Chomsky Normal form

* staircase approach

$X \rightarrow Y_1 Y_2 Y_3 Y_4 Y_5 \ldots Y_{n-2} Y_{n-1} Y_n$

C_1

C_2

C_3

C_4

C_{n-3}

C_{n-2}

$X \rightarrow Y_1 Y_2 Y_3 Y_4 \ldots Y_{n-2} Y_{n-1} Y_n$

can converted to Chomsky Normal form as follows

$X \rightarrow Y_1 C_1$

$C_1 \rightarrow Y_2 C_2$

$C_2 \rightarrow Y_3 C_3$

$C_3 \rightarrow Y_4 C_4$

- - - - - - -

$C_{n-3} \rightarrow Y_{n-2} C_{n-2}$

$C_{n-2} \rightarrow Y_{n-1} Y_n$

$\Rightarrow$ All productions are of type

$V \rightarrow VV$